Lab 4：用户态

3230104220 徐晟洋
2025.12.1

分工说明：队友周皓瑜负责Task1-4，我负责Task5（本来是我写Task4-5的，但是队友为了评测Task1-3，把Task4也写了）

这个实验不大好分工啊

---

1 Part 1：内核对用户态的支持

1.1 Task1：实现进程页表

1.1.1 补全copy_pgd

```c
pgd_t *copy_pgd(pgd_t *old_pgd)
{
    /* Lab4 Task1 */
    pgd_t *new_pgd = (pgd_t *)alloc_page();
    memset(new_pgd, 0, PGSIZE);

    // 模仿free_pgd 太蠢了我先前写的是什么东西啊啊啊
    // 结合 create_mapping 和 free_pgd 完成深拷贝
    // actually 512 = PGSIZE/8

    for (uint64_t i = 0; i < 512; i++) {

        // 取出 L2 pte
        uint64_t old_l2_pte = old_pgd[i];
        if (!(old_l2_pte & PTE_V)) continue;

        if (i >= VA2VPN2(VM_DIRECT_START) &&
            i <= VA2VPN2(VM_DIRECT_END)) {
            // kernel region, just share
            new_pgd[i] = old_l2_pte;
            continue;
        }

        // 找到旧 L1 页表
        uint64_t *old_l1_pgd = (uint64_t *)PA2VA(PTE2PA(old_l2_pte));

        // 分配新的 L1 页表
        uint64_t *l1_pgd = (uint64_t *)alloc_page();
        memset(l1_pgd, 0, PGSIZE);

        // 设置 L2 pte 指向新的页表
        uint64_t l1_pgd_pa = VA2PA((uint64_t)l1_pgd);
        new_pgd[i] = ((l1_pgd_pa >> 12) << 10) | (old_l2_pte & 0x3FF);


        for(uint64_t j = 0; j < 512; j++)
        {
            // 取出 L1 pte
            uint64_t old_l1_pte = old_l1_pgd[j];
            if (!(old_l1_pte & PTE_V)) continue;

            // 找到旧 L0 页表
            uint64_t *old_l0_pgd = (uint64_t *)PA2VA(PTE2PA(old_l1_pte));

            // 分配新的 L0 页表
            uint64_t *l0_pgd = (uint64_t *)alloc_page();
            memset(l0_pgd, 0, PGSIZE);

            // 设置 L1 pte 指向新的页表
            uint64_t l0_pgd_pa = VA2PA((uint64_t)l0_pgd);
            l1_pgd[j] = ((l0_pgd_pa >> 12) << 10) | (old_l1_pte & 0x3FF);


            for (uint64_t k = 0; k < 512; k++)
            {
                // 取出 L0 pte
                uint64_t old_l0_pte = old_l0_pgd[k];
                if (!(old_l0_pte & PTE_V)) continue;

                // 找到旧的数据页
                void *old_page_va = (void *)PA2VA(PTE2PA(old_l0_pte));

                // 深拷贝数据
                void *new_page_va = (void *)alloc_page();
                memcpy(new_page_va, old_page_va, PGSIZE);

                // 设置 L0 pte 指向新的数据页
                uint64_t new_page_pa = VA2PA((uint64_t)new_page_va);
                l0_pgd[k] = ((new_page_pa >> 12) << 10) | (old_l0_pte & 0x3FF);

                get_page(new_page_va);
            }
            get_page(l0_pgd);
        }
        get_page(l1_pgd);
    }
    get_page(new_pgd);

    return new_pgd;
}
```

如图所示，按要求实现copy_pgd。该函数用于创建一个新进程，并加入到进程队列。

1.1.2 修改 proc.c 加入页表相关处理

```
// 直接读取satp值作为初始进程的根页表
pgd_t *p_swapper_pg_dir;
asm volatile("csrr %0, satp" : "=r"(p_swapper_pg_dir));
init_task->pgd = (void *)PA2VA(((long long int)p_swapper_pg_dir << 12LL));
```

设置根页表，作为初始进程，其 pgd 指向内核页表

值得注意的是，对于用户空间 (低地址)，必须深拷贝。需要递归分配新的 PGD/PMD/PTE 页，并为数据页分配新内存并拷贝内容。不能只拷指针，否则一个进程改了内存，另一个进程也会变。

1.2 Task2：实现用户栈与内核栈

1.2.1 设计目的

显然，用户栈与内核栈应该分开。用户态程序使用内核栈不安全，内核处理异常时依赖用户栈不可靠。

所以进入内核（Trap）时：切换到 kernel_sp。

返回用户（Ret）时：切换回 user_sp。

1.2.2 实现思路

1. **PCB 的维护**：在 task_struct 中新增了 kernel_sp 和 user_sp 成员。

   kernel_sp：指向进程内核栈的栈顶。

   user_sp：用于在陷入内核时暂存用户态的栈指针。

2. **Trap 入口的判断逻辑 (_traps)**：利用 sscratch 寄存器区分 Trap 来源。

   若来自用户态：sscratch 中保存的是 PCB 指针。通过 csrrw 交换 tp 和 sscratch，使 tp 指向 PCB，进而获取 kernel_sp 并切换栈。

   若来自内核态：sscratch 为 0（因为进入内核后我们会将其置 0）。此时无需切换栈，直接使用当前 sp。

3. **上下文保存与恢复**：

   保存：将用户态的 sp 保存到 PCB 的 user_sp 中，将通用寄存器保存到内核栈上的 pt_regs 结构中。特别注意 tp 寄存器的保存，需保证存入的是用户态的 tp 值。

   恢复：逻辑的逆过程。关键在于从内核栈恢复到用户栈时，必须确保 sp 是最后恢复的寄存器之一，以防栈指针切换后数据错乱。

1.3 动手做：Trap 来源的判断

要是在 Trap 开头不通过 sscratch 来判断 Traps 的来源，那当 Traps 来自用户态时，还得从 sscratch 里读取 PCB 地址，不然根本没法用 task_struct 切换栈。本来直接用 sscratch 的值就能解决这个问题，再额外用 SPP 就显得多此一举了。

而且，要想保证存储过程里 tp 的值是对的，肯定得有把 tp 写入 sscratch 这一步，这么一来就必须用 csrrw 指令操作 sscratch 了。之所以非用 csrrw 不可，是为了防止原来 tp 的值被覆盖：要是用 csrr 的话，trap 发生前 tp 的值会被冲掉；但用 csrrw 的话，tp 的值会先存进 sscratch，等后续上下文保存的时候，从 sscratch 里把原来的 tp 值读出来再存好就行。

2 Part 2：加载和运行用户态程序

2.1 动手做：查看 `uapp.elf` 的 Program Header

```
root@zju-os /z/code (lab4)# readelf -W -l kernel/user/uapp.elf

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 4 program headers, starting at offset 64

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  RISCV_ATTRIBUT 0x01047d 0x0000000000000000 0x0000000000000000 0x000065 0x000000 R   0x1
  LOAD           0x001000 0x0000000000000000 0x0000000000000000 0x0027a4 0x0027a4 R E 0x1000
  LOAD           0x004000 0x0000000000003000 0x0000000000003000 0x003000 0x003000 RW  0x1000
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10

 Section to Segment mapping:
  Segment Sections...
   00     .riscv.attributes
   01     .text .rodata .srodata.xdigits .eh_frame
   02     .data
   03
```

从 uapp.elf 的 Program Header 能看到，有两个需要加载的 LOAD 部分：第一部分包含 .text、.rodata、.srodata.xdigits、.eh_frame 这些段，

第二部分是 .data 段。

加载时要注意两点：一是地址分配，得把第一部分放到虚拟内存 0x0 的位置，第二部分加载到 0x3000 处；二是权限设置，第一部分要配置成 RE权限，第二部分则设为 RW权限。

2.2 Task 3：实现 ELF 加载器

2.2.1 任务分析

简单来说，就是用户程序以 ELF 格式存在，内核需要解析文件头，将其中的代码段（Text）、数据段（Data）等加载到内存的特定虚拟地址，并赋予正确的权限（如代码段需要可执行，数据段需要可读写）。

2.2.2 实现思路

补全 load_elf_binary 函数：

解析头文件： 首先读取 ELF Header 获取程序头表（Program Header Table）的位置和数量。

段加载： 遍历 Program Headers，对于类型为 LOAD 的段：根据 memsz 申请物理页。将文件内容（filesz）拷贝到内存，不足部分（BSS段）清零。调用 create_mapping 建立映射。必须在权限位中加上 PTE_U，否则用户态无法访问。

用户栈映射： 在用户空间的末尾（高地址）分配页面作为用户栈，并映射为可读写（RW + User）。

2.3 Task 4：运行用户态程序

这部分代码很简单，就是将上述模块整合，实现 user_mode_thread。

顺序如下：

1. 调用 copy_process 创建新进程。
2. 调用 load_elf_binary 加载目标程序。
3. 构造伪上下文 (start_thread)：为了让进程“返回”到用户态，我们手动构造了一个 Trap 上下文。sepc 设置为 ELF 的入口地址（Entry Point）。sp 设置为用户栈顶。sstatus 的 SPP 位设为 0（User Mode），SPIE 设为 1（开启中断）。设置 start_thread 后的返回地址为 ret_from_trap，这样调度器切换到该进程时，会直接跳转到 Trap 返回路径，执行 sret 进入用户态。

不过需要注意的是，这部分代码要临界区保护，否则如果在过程中调度会出大问题，就比如你新建了个进程，还没来得及加载，就开启调度

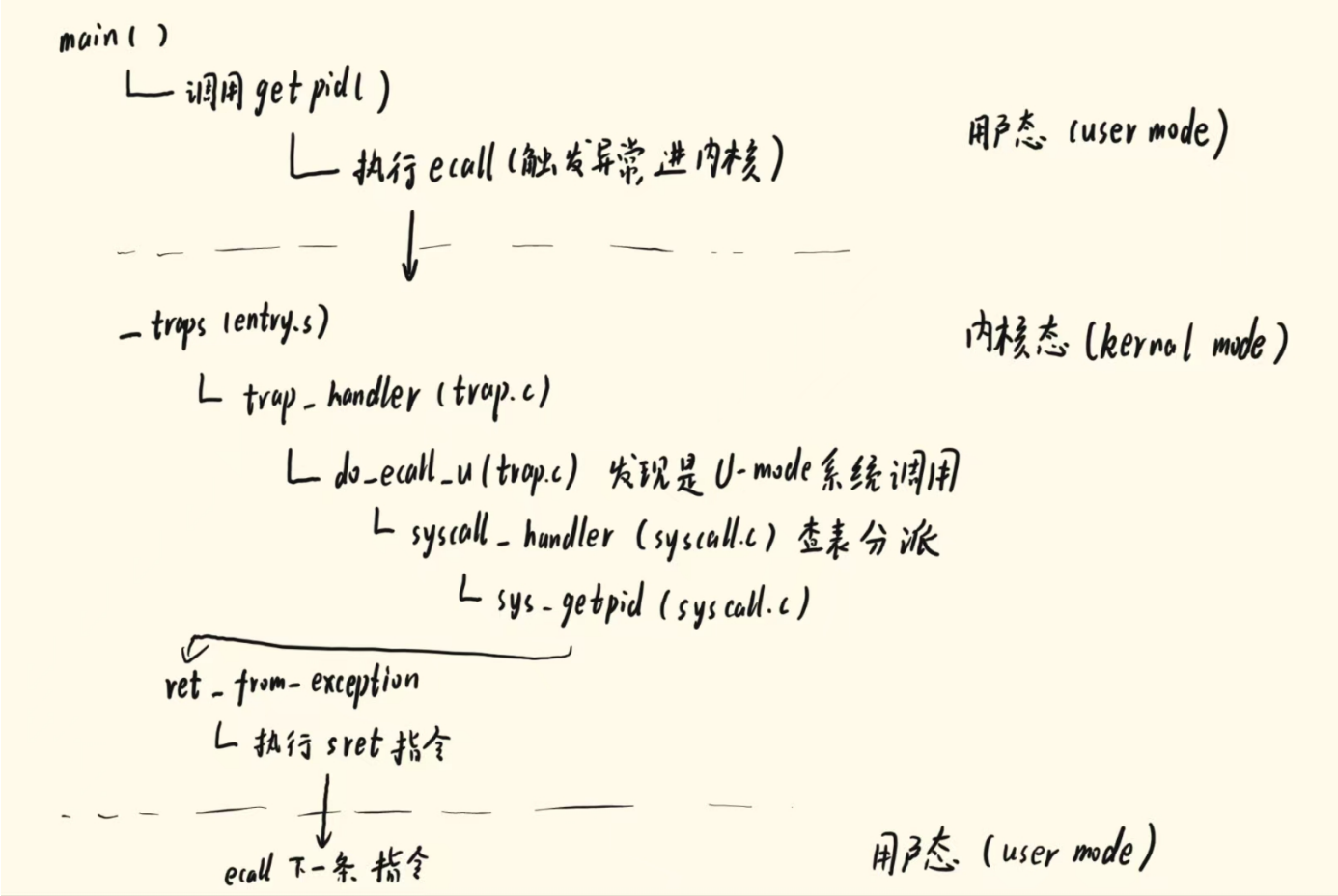
```
struct task_struct *user_mode_thread(void *file)
{
    /* Lab4 Task4 */
    uint64_t flags;
    struct task_struct *new_task;

    // 进入临界区，保存并禁用中断
    interrupt_save(&flags);

    new_task = copy_process(current); //从当前进程拷贝一个新进程

    if (!new_task) {
        // 如果复制失败，恢复中断并返回NULL
        interrupt_restore(&flags);
        return NULL;
    }

    load_elf_binary(file, new_task); //Task3实现，加载用户态程序到新进程的内存空间

    new_task->thread.ra = (uint64_t)ret_from_trap; //设置为 ret_from_trap 即可
    new_task->thread.sp = (uint64_t)task_pt_regs(new_task); //指向初始 Trap 上下文所在的位置，因为 ret_from_trap 会从这里恢复寄存器

    // 退出临界区，恢复中断
    interrupt_restore(&flags);

    return new_task;
}
```

---

3 Part 3：系统调用

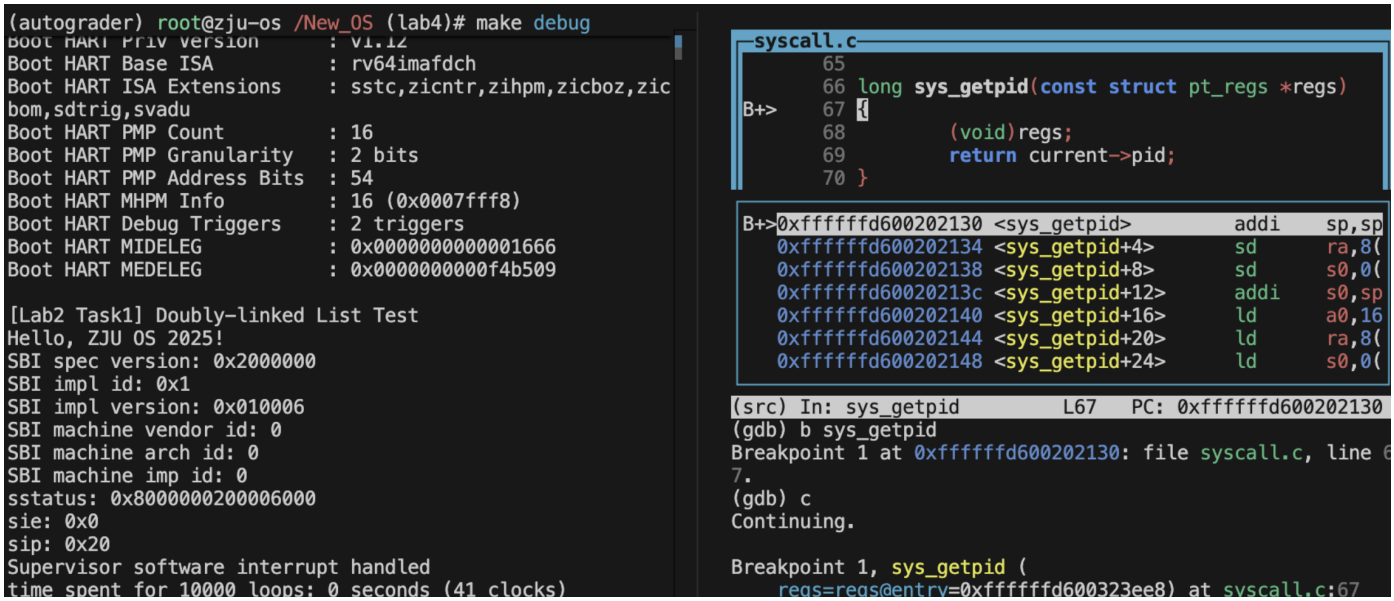
3.1 动手做：系统调用路径

3.1.1 图示



（此处有个笔误，是ret_from_trap）

在系统调用前打断点



3.1.2 栈指针的指向

以下内容展示了调用栈的过程：

1. **save_context (entry.S)**: 负责把当前状况的所有寄存器打包成一个 struct pt_regs。
2. **trap_handler**: 发现是异常。
3. **do_ecall_u**: 发现是“用户态系统调用”异常。

4. **syscall_handler**: 通过查表，发现是 getpid。
5. **sys_getpid**: 正在执行

```
(gdb) bt
#0  sys_getpid (regs=regs@entry=0xffffffd600323ee8)
    at syscall.c:67
#1  0xffffffd600202288 in syscall_handler (
    regs=regs@entry=0xffffffd600323ee8,
    syscall_num=<optimized out>) at syscall.c:77
#2  0xffffffd600202920 in do_ecall_u (
    regs=0xffffffd600323ee8) at trap.c:109
#3  0xffffffd600202a14 in trap_handler (
    regs=0xffffffd600323ee8) at trap.c:128
#4  0xffffffd600200134 in save_context () at entry.S:78
```

下图展示了栈的切换过程

$sp (当前 CPU 寄存器)：显示为高位，即内核空间，说明CPU正在使用内核栈

current->thread.kernel_sp：指向内核栈的栈底（高地址）。每次从用户态进入内核态时，汇编代码都会从这里读取值加载到 sp，从而重置内核栈。

current->thread.user_sp：regs->sp（用户对现场的备份）：显示为低地址，可见系统调用的那一瞬间，用户在使用用户栈。其作用是用于保存用户态的现场。在 _traps 中，用户态原本的 sp 值被交换出来，最终保存在了 struct pt_regs 中（也就是 trap_handler 的参数 regs 里），同时也会更新到 thread.user_sp 以备下次切换。

```
(gdb) p/x $sp
$1 = 0xffffffd600323e78
(gdb) p/x regs->sp
$2 = 0x3fffffffb0
(gdb) p/x regs->sepc
$3 = 0x2f4
```

3.1.3 spec变化

进入traps时为0x2f4，是用户程序代码段的低地址，指向的是执行ecall那条指令的代码

返回用户态时：sepc 变成了 ecall 指令地址 + 4（即 0x2f8）。

这是因为，ecall 是一条指令，不是函数调用。如果不修改 sepc，sret 指令会跳回 sepc 指向的地址（也就是再次执行 ecall），导致程序陷入“陷入内核 -> 返回 -> 再次陷入”的死循环。

3.1.4 参数与返回值的传递 (ABI)

如何传递参数（用户 -> 内核）：

用户态：将参数依次存入 a0 ~ a5 寄存器，将系统调用号存入 a7 寄存器。

内核态：进入 Trap 后，这些寄存器的值被保存在内核栈上的 struct pt_regs 结构体中。内核函数（如 syscall_handler）通过读取 regs->a0 ~ regs->a7 来获取用户传递的信息。

如何传递返回值（内核 -> 用户）：

内核态：系统调用函数（如 sys_getpid）执行完毕后，将返回值写入 struct pt_regs 中的 a0 字段（即 regs->a0 = ret）。

用户态：当执行 sret 返回前，_traps 结尾的汇编代码会将 regs 中的值恢复到真实的物理寄存器中。此时用户的 a0寄存器就拿到了内核给的返回值。

3.2 动手做：“正确”使用 `write` 系统调用

3.2.1 为 `System.map` 符号表中 `_srodata` 符号所在的地址并将其填入

地址是0xffffffd600205000

```
[U] [PID = 1]
pt_regs at %p
binfmt_elf.cELF loaded: entry = %p, PID = %d
started: pid = %lu, fn = %p,

[U] [PID = 3]
pt_regs at %p
binfmt_elf.cELF loaded: entry = %p, PID = %d
started: pid = %lu, fn = %p,

[U] [PID = 3]
 data = %p
kthread.cfinished: pid = %lu, ret = %d
created [pid = %lu, fn = %p, data = %p,

[U] [PID = 1]
 data = %p
kthread.cfinished: pid = %lu, ret = %d
created [pid = %lu, fn = %p, data = %p,

[U] [PID = 1]
task = %p]
Request create [threadfn = %p, data = %p]
kthread_create_infoERRWARN

[Lab2 Task5] Priority-based Scheduling Test

[U] [PID = 3]
task = %p]
Request create [threadfn = %p, data = %p]
kthread_create_infoERRWARN

[U] [PID = 3]
INGINFODEBUGUNKNOWN%s[%s, PID = %lu] %s:%d:%s:

[U] [PID = 1]
INGINFODEBUGUNKNOWN%s[%s, PID = %lu] %s:%d:%s:
```

地址是0x80000000

```
Boot HART ID                : 0
Boot HART Domain            : root
Boot HART Priv Version      : v1.12
Boot HART Base ISA          : rv64imafdch
Boot HART ISA Extensions    : sstc,zicntr,zihpm,zicboz,zicbom,sdtrig,svadu
Boot HART PMP Count         : 16
Boot HART PMP Granularity   : 2 bits
Boot HART PMP Address Bits  : 54
Boot HART MHPM Info         : 16 (0x0007fff8)
Boot HART Debug Triggers    : 2 triggers
Boot HART MIDELEG           : 0x0000000000001666
Boot HART MEDELEG           : 0x0000000000f4b509

[Lab2 Task1] Doubly-linked List Test
Hello, ZJU OS 2025!
SBI spec version: 0x2000000
SBI impl id: 0x1
SBI impl version: 0x010006
SBI machine vendor id: 0
SBI machine arch id: 0
SBI machine imp id: 0
sstatus: 0x8000000200006000
sie: 0x0
sip: 0x20
Supervisor software interrupt handled
time spent for 10000 loops: 0 seconds (70 clocks)

[U] [PID = 1]
```

能使用用户态程序读取出内核中 srodata 部分的值，显然不符合隔离的预期。用户态本不应读取任何内核地址空间或未授权的物理内存；看到的输出是从用户态泄露内核内容。

原因：

共享内核映射：为方便陷入与内核访问，内核页（高地址区域、或 direct-map）被放进了每个进程的页表。

UVA2PA 未做权限与范围校验：仅检查了 PTE_V，就把叶子 PTE 的物理页返回给了 M 态的 sbi_debug_console_write。SBI 在 M 模式直接按物理地址读内存，不再走 S 态页表权限，自然“越权成功”。

可能的解决方法：

sys_write 安全化：跨页分片 + 失败即止

每次最多写到当前页尾，逐页调用 UVA2PA；任意一页无效则返回已写入字节数或错误。

3.3 Task 5：实现 `write` 系统调用

这部分较简单，遍历页表来找到虚拟地址对应的物理地址即可

区别是要在头尾加上检测，即仅当虚拟地址位于用户空间（< VM_START），且叶子 PTE 同时满足 PTE_V | PTE_U | PTE_R 时，才返回物理地址

```
static inline int is_user_vaddr(uint64_t vaddr) {
    return vaddr < VM_START; // 用户地址应低于内核虚拟基址
}

static uint64_t UVA2PA(const void *va)
{
    uint64_t vaddr = (uint64_t)va;
    if (!is_user_vaddr(vaddr))
        return 0;

    uint64_t vpn2 = VA2VPN2(vaddr);
    uint64_t vpn1 = VA2VPN1(vaddr);
    uint64_t vpn0 = VA2VPN0(vaddr);

    pgd_t *pgd = current->pgd;

    uint64_t pte2 = pgd[vpn2];
    if (!(pte2 & PTE_V)) return 0;

    uint64_t *l1 = (uint64_t *)PA2VA(PTE2PA(pte2));
    uint64_t pte1 = l1[vpn1];
    if (!(pte1 & PTE_V)) return 0;

    uint64_t *l0 = (uint64_t *)PA2VA(PTE2PA(pte1));
    uint64_t pte0 = l0[vpn0];
    if (!(pte0 & PTE_V)) return 0;

    // 只允许用户页，且需要读权限（write 从用户读）
    if (!(pte0 & PTE_U) || !(pte0 & PTE_R))
        return 0;
```

```
    return PTE2PA(pte0) + (vaddr & (PGSIZE - 1));
}
```